SIMATS ENGINEERING

PROGRAMMING IN JAVA
ASSIGNMENT 5 — CO5

DATA PERSISTENCE ENGINEERING — FILES AND DATABASES

Student Name	D. Saketh
Register Number	192525018
Course	CSA09 — Programming in Java
JDK	OpenJDK 21
GitHub Repository	Pending creation

GitHub repository is not yet created. Replace the placeholder with the working URL before submission.

Part A — A1: Design a File Format (7 marks)

Text layout: Use delimited fields: passId, register number, name, route number, boarding stop, validity date and amount paid. Escape delimiters/backslashes or use a proper CSV parser. For binary format, document a fixed field order with DataOutputStream.writeUTF for strings, writeInt for route and writeDouble for amount.

Operations: Append with BufferedWriter; find by pass ID and list by route using BufferedReader line scans. Validate each row's field count and types. Record malformed rows in a rejection log. Distinguish missing-file errors. For robust replacement writes, write a temporary file then atomically move it.

Alternative: RandomAccessFile fixed-record lookup is O(1) only if the record index is known; finding by arbitrary ID still needs an index or scan. Fixed-width fields waste padding space.

Part A — A2: Buffered I/O (5 marks)

Prediction: A 50 MiB file contains 52,428,800 bytes, so a one-byte read loop makes about 52.4 million Java read calls. It is slow due to repeated method/native I/O overhead.

Buffered redesign: Use BufferedInputStream with an 8 KiB buffer or BufferedReader for text. Approximate buffer refills: 52,428,800 / 8,192 ≈ 6,400. Both are O(N) time; streaming memory is O(B) for buffer size B.

Alternative: Files.readAllLines() is O(N) time and O(N) memory and creates all lines in memory. It is suitable only when the file comfortably fits in heap. Buffering gives little benefit when reading from an already-memory-resident byte array or when processing a tiny input.

Part A — A3: DAO Pattern (5 marks)

Common pattern: Open connection, prepare/execute SQL, map ResultSet to objects, close resources. Table names, SQL statements, keys and row mapping differ.

Generic interface: Dao should define insert(T), findById(K), findAll(), update(T) and delete(K). StudentDao implements the contract with PreparedStatement and maps Student fields to student columns.

Mapping and alternative: Student.regNo maps to student.reg_no (primary key); name, department and other fields map to corresponding columns. SQL in GUI/service spreads schema dependencies across classes; DAO centralizes changes. Big-O database operation cost is governed by indexes/query plan, not interface abstraction.

Part A — A4: Serialization (4 marks)

Persisted graph: Hostel → Room list → Resident list is traversed recursively. Mark live Connection and password transient; credentials should not be serialized. Ensure all other reachable objects implement Serializable.

Versioning: Declare serialVersionUID. Adding phoneNumber with compatible UID allows older streams to be read, with the new field defaulting to null unless migration/default logic is applied. A UID mismatch causes InvalidClassException.

Alternative: CSV/JSON is portable and easier to inspect but needs explicit parsing and schema evolution. Java serialization is convenient for object graphs but is Java-specific and requires careful version/security handling.

Part A — A5: Transactions and SQL Injection (4 marks)

Transaction pseudocode: Set autoCommit(false); debit source wallet; credit destination wallet; commit only after both updates succeed; on any exception rollback and rethrow; restore autoCommit in finally.

Injection: Concatenating user input into SQL can let a crafted value such as `' OR '1'='1` change the WHERE condition. Use `SELECT ... WHERE id=? AND pwd=?` and bind values with PreparedStatement.

Alternative: Two auto-commit updates can leave a debit without a matching credit if the second fails. A transaction adds coordination/commit overhead but preserves atomicity and correctness.

Part B — B1: Instrumented File Store (7 marks)

Implementation: PassStore.java appends CSV records using BufferedWriter and searches line-by-line using BufferedReader inside try-with-resources. It skips rows with the wrong number of fields or invalid numeric route/amount.

Tests: The demo adds P1001 and P1003, injects a damaged P1002 row, finds P1003 and attempts P1002. Expected and actual output are [P1003, 192311003, Divya, 7, 3800.0] and null.

Buffering: LineCounter.java compares an unbuffered byte read with BufferedReader on a small fixture. The requested 1, 10 and 50 MB timing runs and 1,000–100,000 record growth measurements remain to be recorded on the student's machine; small fixture output is not a substitute for the required benchmark.

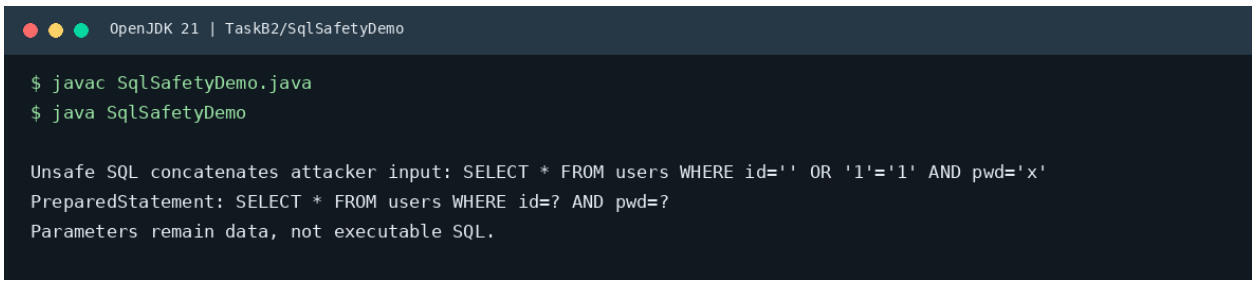


Part B — B2: Adversarial Inputs (5 marks)

Unsafe and safe SQL: The demo prints an unsafe concatenated query with a crafted input and contrasts it with a parameterized PreparedStatement template. The parameterized version treats the payload as data.

File adversarial cases: PassStore ignores damaged field-count rows and invalid numeric values while continuing to scan. Extend the test fixture with empty lines and non-numeric amount values and record each observed result.

Limit: This environment did not have a configured MySQL test database, so a live unsafe-login acceptance versus PreparedStatement rejection was not claimed. Run the supplied examples against a local assignment-only database.



Part B — B3: Statement vs PreparedStatement vs Batch (8 marks)

Implementation: PassLoader.java uses PreparedStatement, binds the five pass fields, batches 1,000 rows, commits once, and rolls back on SQLException or RuntimeException. SQL schema is supplied in sql/schema.sql.

Execution status: The JDBC source compiles against the Java standard library. It requires a MySQL JDBC driver, a running test database and connection configuration to execute. No 1k/10k/100k timing or transaction failure measurements are fabricated.

Expected trade-off: PreparedStatement avoids repeated SQL parsing and safely binds values; batching reduces round trips. A single transaction allows rollback of all inserts after a mid-batch failure. Single counter issue can use one PreparedStatement; large nightly import should use batch + transaction.

Part B — B4: Final Engineering Decision (5 marks)

Migration strategy: Read each legacy pass record, validate field count and types, record rejected rows, then load valid records in batches inside a transaction. The included MigrationDecision.java demonstrates input validation and a rejection report.

Metadata: DatabaseMetadataReport.java uses DatabaseMetaData to print product, driver version and table names when JDBC_URL, JDBC_USER and JDBC_PASSWORD are configured.

Production decision: For 12 campuses and 100× growth, a shared relational database is the strongest fit for concurrent counters, indexed searches, constraints and transactional safety. Text files are useful for import/export; Java serialization is less suitable for multi-user shared updates.

Limitation: A live MySQL migration, metadata run and measured production comparison require the target database, driver and credentials. The current execution proves file validation and Java compilation, not database connectivity.

